Appendix O — Original detailed week plan — per-day reading and self-checks (use with Part I §3)

Source: `GITLAB_WEEK_PLAN.md`

GitLab Agentic SDLC — One-Week Prep Plan (Oct 2 → Oct 9, 2026)

Interview: Friday 2026-10-09, Karan Bhatia (VP Engineering & Engineering Fellow, GitLab). **Content lives in:** `GITLAB_KARAN_BHATIA_DEEP_DIVE.md` (referred to below as **DD §n**). **This file is the schedule:** what to read, what to write from memory, what to say out loud, and how to check yourself each night.

Daily shape: ~6–8 focused hours on full days, in 90-minute blocks with breaks. Each day ends with a self-check you answer **out loud, without notes**, recorded on your phone if possible. Anything you can't answer cleanly goes on a running `GAPS.md` list that Oct 7–8 drain.

Before anything: how this interviewer probes, and how that changes your prep

Who he is (from his LinkedIn and his own writing — nothing beyond this):

- **~2 months into GitLab** (since Sep 2026) as VP Engineering & Engineering Fellow. He is still ramping on GitLab's codebase himself. He won't gatekeep on Rails internals; he *will* test fundamentals, judgment, transferable mental models, and your own systems in depth.
- **New Relic, 2022–2026:** Principal → Senior Principal Architect → VP Eng & Technical Fellow. Platform architecture across Security, Billing, Identity and Access Management, Growth. Self-described expertise: **"scaling multi-tenanted systems."** Recent interests: **observability as the control system for autonomous agents**; New Relic Lens (federated queries across telemetry and business data); the **New Relic × GitHub Copilot closed loop** (regression detected → issue filed → coding agent drafts a fix → PR for human review → post-merge validation) — which is this job's mandate in miniature; NRDB-scale data systems.
- **Microsoft Azure Active Directory, 2016–2022:** 5.5 years of **identity and access management at cloud scale**.
- **Accenture Federal, 2015–16:** operated CI pipelines and AWS automation.
- **BS CS & Math (Virginia Tech, Phi Beta Kappa); MBA (UW).**
- **His writing voice** ("Observability in a distributed world," New Relic, Aug 2024): short, concrete, code-first. Propagate `traceId`, `upstreamApplication`, `userId`, `tenantId` across every hop; examples in Rails and Spring; a root cause found by following one `traceId` with a query.

What a VP + Fellow probes that a peer screen doesn't (GitLab's handbook: the Fellow "solves technical problems of the highest scope and complexity for the entire organization"):

1. **Where your mechanism stops working** — and at what scale. Every answer needs its own failure mode attached.
2. **How you reason when you don't know.** Naming the unknown and the experiment beats a confident guess. Bluffing is the one unrecoverable error.
3. **Numbers with methods.** "86%" invites "of how many, what interval, what baseline, what did the misses look like, how would you monitor it in production."
4. **Assumption removal.** He'll design with you, then take away the human approval, or multiply tenants by 100, or make the issue author hostile, and watch you re-derive.

5. **For him specifically:** IAM follow-ups (which token, minted by whom, audience, lifetime across a pause, revocation, audit), observability follow-ups (which attributes are on the span, what's the query), and multi-tenant fairness/isolation.

How that changes your prep this week:

- Less memorizing GitLab internals, more **deriving from first principles out loud**.
- Every resume number gets a **method** (DD §2.1 template) — and any claim you can't back with an artifact gets fixed or reframed **before** Friday (DD §0.3).
- Lead with your **three bridges to his career** (DD §0.2): identity propagation, observability for agent correctness, multi-tenancy — then the agent architecture.
- Practice answers in his register: **mechanism → example (code or query) → failure mode → the signal you'd watch.**
- Ruby: **conversational, not fluent** — enough to read and reason about a Rails controller, service, model, and Sidekiq worker like the snippets in his own post.

The CrowdStrike lesson, and the infra track that runs all week

The CrowdStrike round (2026-09-17) did not pass. Agent architecture and security landed. **Direct questions on AWS deployment and Salesforce Data Cloud did not**: there wasn't genuine depth you could defend. This round is *more* exposed on that axis, because your resume claims AWS/Docker/K8s/CI-CD/Splunk operation and on-call, the JD asks for it, and he is a platform/observability architect. So infrastructure runs as a **parallel track all week**, not one squeezed paragraph:

Day	Infra-track work	Where
Fri Oct 2	Read DD §0's flag and DD §14.1; pre-rate every row by gut (OWN / PATTERN / THIN)	evening, 30 min
Sat Oct 3	Your public OWN example: ResolveFlow CI/CD + Docker + Render/Vercel, with its gaps (DD §14.3)	Block 6, 45 min
Mon Oct 5	**Office-Claude §A (AWS) + §B (CI/CD)** against your real environment; Data Cloud re-read (ANIRUDDHA §1.4/§1.4a, DD §14.6)	lunch + Block 6, ~2 h
Tue Oct 6	**Office-Claude §C (Kubernetes) + §D (Splunk + incident)**; rehearse the DD §14.5 worked trace with your real system names	Block 6, ~1.5 h
Wed Oct 7	**Dedicated infra-depth block (2.5 h)**: DD §14 in full, ANIRUDDHA §3 with fresh eyes; **office-Claude §E (Data Cloud) + §F (40% / on-call)**; finalize §14.1 ratings	Block 0
Thu Oct 8	Mock includes ≥3 infra probes asked as pivots right after strong agent answers (that's how the CrowdStrike round turned)	Mocks 1–2
Fri Oct 9	Re-read your filled-in DD §14.1 table and the boundary sentences	morning

Prompts for your work environment: `OFFICE_CLAUDE_QUESTIONS.md`. The output of each session goes straight into the Rating column of DD §14.1.

Fri Oct 2 — Kickoff & honesty audit (partial day, ~3–4 h)

Theme: know exactly what you're defending before you defend it.

Block 1 (60 min) — orient

- Read DD §0 entirely (0.1–0.4). Read the job description once more, slowly; highlight every verb ("build," "develop and deploy," "measure and improve," "debug," "mentor").
- Skim DD's table of contents so you know where everything lives.

Block 2 (75 min) — audit your own repo against your resume (DD §0.3)

- In `~/Desktop/ResolveFlow`: `uv sync`, then `uv run python -m eval.harness`. Write down the exact pass count it prints (DD §0.3 R9 — README says 12, the task files suggest 11).
- Open `eval/` and confirm for yourself: **is the 50-scenario, multi-provider benchmark behind "86% routing accuracy at 24% lower cost" anywhere you can show?** (DD §0.3 R8.) Check other machines, notebooks, branches, Google Drive.
- **Decide tonight, in writing**, which of the three paths you take for R8: (a) it exists → commit it with its results file by Oct 6; (b) it doesn't → build it Sun evening/Mon evening from the spec in DD §4.7.6 and report whatever it actually produces; (c) neither → rehearse the honest framing in DD §0.3 R8 until it's natural. **Recommendation: (a) or (b).** He may open your GitHub; a claim you can show is an asset, one you can't is a liability.
- Note R4 (code evidence / `owner_hint` / `files_touched`), R10 (daily-limit mismatch), and R11 (the 0.35 threshold comment's arithmetic, DD §3.5) on `GAPS.md`.

Block 3 (60 min) — re-read your own post, first pass

- Read `pdf/medium_post.md` end to end. This is the single best prep asset you have: it is, section by section, what GitLab's orchestration service is supposed to be (DD §0.4).
- Mark the six lines in DD §0.4 you want verbatim. Mark any claim in the post you couldn't defend under a "why?" — those go on `GAPS.md`.

Block 4 (30 min) — the numbers sheet

- Start `NUMBERS.md`: one row per resume number (92%, 60→5 days, 150+ users/8 teams, 20→3 min, 35%, 40%, 10+ teams, 5+ engineers, 2,750 chunks, 0.35, 50 scenarios, 86%, 24%, +12%, –21%, –18%, 100 questions, 90→15 min, 70%, 25%, 30%, 18%, 40%, 30%). For each: the five questions from DD §2.1. Fill what you can tonight; the rest on Oct 3.

Block 5 (30 min) — infra-track kickoff

- Read DD §0's ⚠️ flag and DD §14 intro + §14.1. Pre-rate each row I1–I14 by gut. Wherever your gut says THIN on something your resume implies you OWN, that's this week's priority list.
- Skim `OFFICE_CLAUDE_QUESTIONS.md` so you know what you'll be running Mon–Wed. If you have work-environment access over the weekend, start §A early.

End-of-day self-check (out loud):

1. Which resume claims can be checked against public code, and what will he find?
2. What's your decision on R8, and what's the first concrete step tomorrow?
3. Say your post's "probabilistic core, deterministic shell" principle in two sentences.
4. Which three infra rows in DD §14.1 scare you most, and why?

Sat Oct 3 — Your own systems at implementation depth (full day)

Theme: "show me the code," "what breaks at scale," "why this and not X" — for everything you built. This is the day that most determines the outcome.

Block 1 (90 min) — ResolveFlow, file by file

- With DD §2.3 open, read every file in the repo map in order: `graph/state.py` → `graph/build.py` → each node → `tools/github_client.py` → `tools/retrieval.py` → `ingest.py` → `app/main.py` → `app/db.py` → `eval/harness.py` + `eval/tasks.py` → `eval/capability_harness.py` → `.github/workflows/*.yml` → `docs/CHECKPOINTING.md`.
- For each file say out loud: what it does, why it's shaped that way, what breaks.

Block 2 (90 min) — write from memory, no reference

- On a blank file, write: (1) the `GraphState` TypedDict; (2) the three conditional-edge maps from `build.py`; (3) `await_approval` with `interrupt()`; (4) `execute` with the `PermissionError` guard and the label `try/except`; (5) `independent_review`'s three gate booleans, including the `bool(citations) and all(...)` subtlety; (6) the FastAPI `analyze` and `resume` handlers including the `thread_owners` 403 check.
- Diff against the real code. Every mismatch → `GAPS.md`.

Block 3 (75 min) — what breaks: DD §2.4 F1–F9

- For each finding, explain the bug and the fix aloud in under 90 seconds.
- **Implement two of them for real** (they're small and make great talking points): F1 (per-thread lock or unique-key claim on resume) and F7 (approval TTL + re-validate the issue is still open at resume). Optional: F4's GitHub App migration as a written design note, not code.
- Practice the IAM self-critique (DD §0.2 Bridge 1, "own the weak spot"): `public_repo` scope, plaintext token, shared fallback, token in checkpoint state (DD §4.4).

Block 4 (90 min) — Salesforce bullets S1–S7 (DD §2.2)

- Re-read `ANIRUDDHA_SINGH_DEEP_DIVE.md` §1.2–1.7 for the internals (supervisor/specialists, Reasoner/Executor, Skills/NLDs, generic actions, token-exchange isolation).
- For each bullet in DD §2.2, write the 30-second answer and the methodology (fill `NUMBERS.md`). Especially:
- **S1:** the "access management isn't SDLC" answer — rehearse until it's calm and short.
- **S3 (Slack agent):** the full IAM chain — signing-secret verification, 3-second ack + event-id dedupe, Slack-user → enterprise-identity mapping (be exact about what you actually did), whose token executes, token lifetime vs. approval wait, revocation. He spent 5.5 years on Azure AD; this is where he'll dig.
- **S4:** the eval framework as layers + two bars + replayed failures, then reframed as observability (DD §0.2 Bridge 2).

Block 5 (60 min) — other projects (DD §2.5)

- AdaptiveRAG: pin down relative vs. absolute for +12%, how groundedness was scored, why adaptive is cheaper; the n=100 interval and McNemar point.
- Bug Triage skill: n, who timed it, why read-only by construction.
- Disney decoder blocklist as an observability-driven control loop.

Block 6 (45 min) — your public CI/CD and deploy story (DD §14.3, §14.1 I4/I6)

- Re-read `../.github/workflows/deploy.yml`, `../Dockerfile`, `../render.yaml`, `../vercel.json`.
- Say it out loud in 90 seconds: trigger → `verify` (safety-gate evals) → `needs: verify` → Vercel + Render deploys → notification. Then the gaps, each with its fix: no staging/canary/ automated rollback; deploy jobs `exit 0` when secrets are missing; the Render hook isn't tied to the verified SHA; the hook key sits in a URL; **the Dockerfile isn't what runs on Render** (`render.yaml` uses the native Python runtime).
- ResolveFlow is **not on AWS**. Know that cold, and never let it be implied otherwise.

Evening (optional, 60–90 min) — R8 path (a)/(b) start: if building the benchmark, write the scenario JSON schema and the first 15 scenarios (DD §4.7.6).

End-of-day self-check (out loud, no notes):

1. Walk ResolveFlow from issue URL to posted comment, naming every node, every state key it writes, and the one place a side effect happens.
2. What runs twice when you resume from `interrupt()`, and why is ResolveFlow safe today?
3. Two simultaneous resume requests arrive — what happens, and what's your fix?
4. "Your agent runs with the user's permissions" — which token, minted how, with what scope, stored where, and what would you change?

5. 60 days → 5 days: what exactly was timed, median or mean, and what share is the agent vs. the process change?
6. ResolveFlow's deploy pipeline, from push to live, with three gaps and their fixes.

Sun Oct 4 — Ruby/Rails crash course + GitLab's monolith (full day)

Theme: be conversational in Ruby and Rails — able to read, follow and reason about a controller, service, model and Sidekiq worker like the snippets in his own blog post — and know how GitLab's runtime fits together. Not fluency; literacy plus pattern-mapping.

Block 1 (90 min) — Ruby syntax for a Python engineer

- Read: Ruby in 20 Minutes — https://www.ruby-lang.org/en/documentation/quickstart/
- Read: To Ruby From Python — https://www.ruby-lang.org/en/documentation/ruby-from-other-languages/to-ruby-from-python/
- Then DD §11.1. Install Ruby locally (`brew install ruby` or rbenv) and type every line of DD §11.1 into `irb`. Specifically test: truthiness (`0` and `""` are truthy), symbols vs. strings as hash keys, blocks with `map/select/each_with_object`, `&:method`, `&.`, `?/!` method conventions, keyword args, `attr_reader`, modules as mixins, `begin/rescue/ensure/retry`.
- Exercise: rewrite ResolveFlow's `classify()` and `IssueEvidence.is_information_sparse` in Ruby (≈20 lines). Don't polish — just make it run in `irb`.

Block 2 (90 min) — Rails MVC + ActiveRecord mental model

- Read: Getting Started with Rails — https://guides.rubyonrails.org/getting_started.html (skim the scaffolding; focus on routes → controller → model → view flow).
- Read: Active Record Basics — https://guides.rubyonrails.org/active_record_basics.html
- Read: Active Record Query Interface, the **"Locking Records for Update"** section (optimistic `lock_version` and pessimistic `lock`/`with_lock`) — https://guides.rubyonrails.org/active_record_querying.html
- Then DD §11.2. Exercise: for each of the four concurrency snippets in DD §11.2 (`lock_version`, `with_lock`, `create_or_find_by!`, transaction + outbox row), write the equivalent SQL and the Python/psycopg equivalent from your own `app/db.py` style.

Block 3 (75 min) — Sidekiq and GitLab's worker conventions

- Read: Sidekiq Best Practices — https://github.com/sidekiq/sidekiq/wiki/Best-Practices (idempotency, small simple args).
- Read: GitLab's Sidekiq development guide — https://docs.gitlab.com/development/sidekiq/ (find: `idempotent!`, deduplication strategies, `data_consistency`, `urgency`, `feature_category`; verify the exact names against DD §11.3).
- Then DD §11.3–11.4. Exercise: read the illustrative `PublishMergeRequestWorker` and service in DD §11.4 and annotate each line with the distributed-systems concept it implements (fencing, authz at execution time, find-or-create idempotency, optimistic locking, read-your-writes). Then explain it aloud in 5 sentences.

Block 4 (90 min) — skim GitLab's monolith on GitLab.com

- https://gitlab.com/gitlab-org/gitlab — browse (paths move; search if a path 404s):
- `app/services/merge_requests/` — open the create service; read one `execute` method end to end. Note `ServiceResponse`.
- `app/policies/project_policy.rb` — skim how abilities are declared (DeclarativePolicy).
- `app/workers/` — open three workers; find their attribute declarations.
- `lib/api/` — one Grape endpoint (e.g., merge requests).
- Search the repo for `duo_workflows` — find the Rails-side models/services/API for workflows and checkpoints (likely under `ee/`).

- Read: GitLab architecture overview — https://docs.gitlab.com/development/architecture/ (Workhorse, Puma/Rails, Sidekiq, Gitaly, Praefect, Runner, Postgres, Redis).
- Then DD §11.6. Exercise: **draw DD §11.6's runtime diagram from memory** and mark where a remote agent flow runs, where its checkpoints go, and where its git operations happen.

Block 5 (45 min) — his idiom

- Find and skim his New Relic post "Observability in a distributed world" (newrelic.com blog; search the title). Then DD §11.5: write the agent-run version of his Rails `add_custom_attributes` snippet from memory.

Evening (optional, 60–90 min) — R8 path (b): finish scenarios, wire a thin provider abstraction (OpenAI + one other), run the baseline.

End-of-day self-check:

1. Read this aloud and explain it: `run.with_lock { run.update!(state: :publishing) if run.gated? }`. What SQL runs? What must never go inside the block?
2. Why is `find_or_create_by` racy and what fixes it?
3. A Sidekiq job runs twice. Which three worker-level and data-level mechanisms make that safe?
4. What does `data_consistency :always` buy you, in terms of the read-your-writes problem?
5. Draw GitLab's runtime and say where "deterministic execution" lives in it.
6. Say the one-paragraph Ruby-gap framing from DD §0.2b naturally, once.

Mon Oct 5 — GitLab's product and vocabulary day (full day)

Theme: speak GitLab's language back to them; understand what they've actually built; form opinions you can defend. He's ramping too — you're aiming for informed and opinionated, not encyclopedic.

Block 1 (90 min) — the Duo Agent Platform, from primary sources

- https://docs.gitlab.com/user/duo_agent_platform/ — agents, flows, AI Catalog, MCP, external agents. Write down every foundational flow name.
- https://docs.gitlab.com/user/duo_agent_platform/security/ and https://docs.gitlab.com/user/duo_agent_platform/composite_identity/ — composite identity, `ai_workflows/mcp` scopes, intersection semantics, MR attribution for segregation of duties.
- https://docs.gitlab.com/user/duo_agent_platform/security_threats/ — lethal trifecta, prompt-injection detection modes (no checks / log only / interrupt), reader/writer split, per-environment controls table (note: remote flows have no per-call human approval).
- Then DD §1.2 and DD §1.8. **Verify every fact in DD §1.2**; mark anything that changed.

Block 2 (75 min) — the runtime architecture

- https://runbooks.gitlab.com/duo-workflow-svc/ — LangGraph, checkpoints saved to GitLab, Runway/Cloud Run, LangSmith, alerts (checkpoint error rate, LLM error rate), the executor WebSocket 1006 failure mode.
- https://handbook.gitlab.com/handbook/engineering/architecture/design-documents/duo_workflow/ and its decision records (ADR 001 "Move Duo Workflow Service into the AI Gateway", ADR 004 "Workhorse as a Duo Workflow Service proxy").
- Then DD §4.4's last paragraph (why checkpoints in Rails) and DD §9.3–9.4.
- Exercise: **whiteboard the Duo Workflow topology** (IDE/CI executor ↔ gRPC ↔ Workflow Service (LangGraph) ↔ AI Gateway/models; Workflow Service ↔ Rails for checkpoints and API; Workhorse in the path) and annotate where you'd put the `traceId` and how it crosses each hop.

Block 3 (60 min) — Act 2 and governance

- Re-read the Act 2 CEO letter (Bill Staples, 2026-05-11; about.gitlab.com blog) and the governance post on tool-approval guardrails (always allow / ask / deny).
- Map each Act 2 phrase to a DD section: "assigning work, managing state, passing context, resolving conflicts, enforcing policy, human in the loop" → DD §4.3, §4.4, §4.9, §4.5; "agent-rate commits / Git re-engineered" → DD §5.1, §6.6; "governance as core platform services" → DD §7.
- Write **your opinion** (one paragraph each) on: (1) remote flows without per-call approval (DD §7.4); (2) log-only as the default injection-detection mode (DD §7.3); (3) checkpoints stored in Rails vs. a service-local store (DD §4.4).

Block 4 (75 min) — CI/CD and Git mechanics that matter at agent rate

- Merge trains — https://docs.gitlab.com/ci/pipelines/merge_trains/
- Protected branches and protected CI/CD variables (docs.gitlab.com — search "protected variables"); CI job token scope (search "CI/CD job token").
- GitLab Runner overview — https://docs.gitlab.com/runner/ (executors, how jobs are picked up).
- Gitaly / Gitaly Cluster overview — https://docs.gitlab.com/administration/gitaly/
- Then DD §6.2, §6.6, §7.2.

Block 5 (60 min) — his reference architecture

- DD §1.10 (the closed loop he championed) — draw it from memory twice. Rehearse the five failure modes and the "where your systems sit on the loop" table.
- Write the answer to: "How would you build the New Relic × Copilot loop natively inside GitLab?" — 2 minutes spoken.

Block 6 (~2 h, split lunch + evening) — infra track: AWS, CI/CD, Data Cloud

- **Office-Claude §A (AWS) and §B (CI/CD)** in your work environment, using the preamble. Record VERIFIED vs INFERRED. Fill DD §14.1 rows I1–I3, I7, I8.
- Then DD §14.2 (Lambda vs ECS vs API Gateway) and §14.3 (pipeline mechanics, canary vs blue- green, rollback). Write your real Slack-agent deploy path from merged PR to prod on one page, every stage and gate, with what you OWN marked.
- **Data Cloud re-read, fresh eyes:** ANIRUDDHA §1.4 and §1.4a, then DD §14.6. Say the 8 ingestion stages and the freshness numbers (10–15 min CRM ingestion; 1–24 h identity resolution; ACL fields only as fresh as the last re-embed) from memory. Practice the "if asked X, say Y" table aloud. Pay most attention to the first row, which is your honest consumed-vs-configured answer.

End-of-day self-check:

1. What is composite identity, exactly, and what are three IAM follow-up questions an Azure AD veteran would ask about it? (DD §7.1)
2. Where do Duo Workflow checkpoints live, and what's one trade-off of that choice?
3. Name eight foundational flows.
4. What's the lethal trifecta, and how does each GitLab control break one leg?
5. Explain merge trains in 45 seconds and say what fails at agent rate.
6. Your opinion on remote flows having no per-call approval — 60 seconds, with the risks.
7. "Tell me about your AWS deployment experience." 2 minutes: role first, then depth on what you owned, then design reasoning for the rest.
8. "Have you worked with Data Cloud?" Your exact consumed-vs-configured answer, then the pipeline.

Tue Oct 6 — Agentic framework depth: orchestration, durability, guardrails, evals, observability

Theme: the vendor-neutral mechanics, at the level where you can write them live.

Block 1 (90 min) — LangGraph internals + the MR agent from memory

- LangGraph docs (langchain-ai.github.io/langgraph — now hosted at docs.langchain.com): read the pages on **persistence/checkpointers**, **interrupts / human-in-the-loop**, **durable execution**, and **Send / Command**.
- DD §4.4 in full. Then: **write DD §1.9's MR-authoring graph from memory, twice**, no reference. Second time under 25 minutes. Diff against the doc.
- Explain aloud: supersteps, reducers, `aput_writes`, why pre-`interrupt()` code runs twice, what the checkpointer does NOT give you (four items).

Block 2 (75 min) — orchestration patterns, HITL, guardrails

- DD §4.1–4.3, §4.5, §4.6. Cross-read `study_guide_enterprise_system_v2.md` §5.3–5.8 (confused deputy, injection, hard/soft guardrails, gateway, HITL tiers) — don't re-learn, connect.
- Exercise: for each row of DD §4.6's guardrail table, answer "can the model talk its way past this?" out loud.
- Write DD §4.13(a) (artifact policy decision) from memory.

Block 3 (90 min) — evaluation frameworks for code agents

- DD §4.7 entirely. Read the SWE-bench overview (https://www.swebench.com/) for the fail-to-pass / pass-to-pass framing.
- Exercise: compute the Wilson interval for 43/50 by hand (DD §4.7.4) and explain McNemar in one breath. Then design the eval for the test-generation agent (DD §8.2) on paper in 20 min.
- If on R8 path (b): run the benchmark today, commit results, and write the README section. Prepare to discuss the confusion matrix — what the misses looked like.

Block 4 (75 min) — observability for agents, in his vocabulary

- DD §4.8 entirely. Skim OpenTelemetry GenAI semantic conventions — https://opentelemetry.io/docs/specs/semconv/gen-ai/
- Exercise: write the span attribute schema from memory (traceId, upstreamTrigger, tenantId, requestingUserId, actingIdentity, threadId, checkpointId, node, manifest fields, tool, policy decision, cost). Then write three trace queries: "why did the agent open this MR", "which tenants hit policy denies today", "did resolve rate drop after the model change."
- Rehearse the two-sentence answer at the end of DD §4.8 until it's natural.

Block 5 (45 min) — security for code-pushing agents

- DD §4.11 and §7.3. Read Simon Willison's "The lethal trifecta" — https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/
- MCP authorization at a skim: https://modelcontextprotocol.io/specification (find the authorization section; note audience-bound tokens and no token passthrough).

Block 6 (~1.5 h) — infra track: Kubernetes + Splunk incident

- **Office-Claude §C (Kubernetes) and §D (Splunk + incident reconstruction).** Fill DD §14.1 I5, I9, I10.
- DD §14.4: say the K8s primitives in one line each; walk a pod from image push to serving to graceful shutdown; explain why queue-driven agent workers should scale on queue depth, not CPU.
- DD §14.5: rehearse the worked trace with **your real system names and your real D6 incident**. Do it twice, out loud, each time in under 4 minutes, in his blog post's shape: symptom → traceId → follow it across services → root cause in a dependency → fix → regression.

End-of-day self-check:

1. Write `await_approval` with hash-binding of the approved artifact. Explain why the hash.
2. pass@k vs pass^k — which for what, and what's 0.99^10?
3. 43/50: what's the 95% interval, roughly? How do you compare two models on the same 50?

4. What attributes are on every span of an agent run, and how does the trace survive a six-hour approval pause?
5. Name nine threats to a code-pushing agent and the structural control for each.
6. Recite DD §4.14's lessons 1–12.
7. A pod is OOMKilled vs CPU-throttled: what's the difference, which setting causes each, and what happens to an in-flight agent step?
8. Tell your real incident (D6) in 3 minutes with timestamps and the queries you ran.

Wed Oct 7 — Distributed systems at agent rate, scalability, and system-design whiteboards

Theme: raw scale judgment (he respects NRDB-class thinking) and live design under pressure.

Block 0 (2.5 h, first thing) — dedicated infra-depth block (the CrowdStrike gap)

- Read DD §14 in full, start to finish, slowly.
- Re-read ANIRUDDHA §3 (scalability: action-catalog funnel, centralized-retrieval scaling risk, "Thinking…" latency, serial-execution bottleneck, long-tail cost) **with fresh eyes**. For each subsection ask: what did *I* touch here, and what do I only know about?
- **Office-Claude §E (Data Cloud for my systems) and §F ("~40%" and on-call methodology).** These two are the highest-stakes sessions of the week. Fill DD §14.1 I11–I13.
- **Finalize every Rating in DD §14.1.** For each THIN row, write the one boundary sentence and the first-principles answer you'll give.
- Exercise: answer aloud, timed, 2 minutes each: (1) "Walk me through how your agent service gets to production and how you'd roll it back." (2) "What does 'throughput up 40%' mean, and how was it measured?" (3) "How fresh is the data your agent retrieves from Data Cloud?" (4) "Describe your on-call rotation and your worst page."

Block 1 (90 min) — distributed mechanics

- DD §6 entirely. Read Kleppmann's fencing-token argument — https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Cross-read `ANIRUDDHA_SINGH_DEEP_DIVE.md` §2.8 (concurrency rules) and §4 (applied distributed mechanics) — connect, don't re-learn.
- Exercise: write DD §4.13(e) lease-with-fence SQL from memory; explain the zombie-worker scenario it prevents.

Block 2 (60 min) — scalability with numbers

- DD §5 entirely. **Redo the DD §5.1 back-of-envelope from scratch** with different assumptions (e.g., 200k runs/hour, 2 pipelines/run, 15 min each) and find the bottleneck. Do it out loud in under 4 minutes.
- Write DD §4.13(d) token bucket and explain weighted fair queuing across tenants (DD §5.3).

Block 3 (2 × 45 min) — timed whiteboards

- 45 min: DD §8.1 (500 agents in one monorepo). Timer on; talk out loud; draw on paper or a whiteboard app. Then compare with the DD outline; note what you missed.
- 45 min: DD §8.4 (his closed loop). Same protocol.

Block 4 (2 × 30 min) — shorter whiteboards

- 30 min: DD §8.3 (security remediation guardrail).
- 30 min: DD §8.5 (multi-tenant platform for .com and self-managed).

Block 5 (45 min) — on-call scenarios in his register

- DD §9 entirely. For 9.2–9.7, answer each aloud as: alert → query → narrow → root cause → fix → regression case. Write the query each time.

Block 6 (30 min) — Go refresher

- Type DD §4.13(f) (Go worker pool) from memory; then critique it the way DD says (in-process map vs. fleet-wide serialization). The JD lists Go; GitLab Runner, Gitaly and Workhorse are Go — be able to read and write a simple concurrent Go program.

End-of-day self-check:

1. At 50k runs/hour, what's the binding constraint and why? What's the cheapest lever?
2. One bad CI template breaks 300 pipelines with the fix-CI flow on. What happens and what should happen?
3. Explain fencing tokens and when you need one.
4. Which reads must go to the primary, and how does Sidekiq express that?
5. How do merge trains behave when 10% of agent MRs fail, and what do you do about it?
6. Read your completed DD §14.1 table. Is any row still rated OWN that a 3-question follow-up would expose as PATTERN? Fix it now.

Thu Oct 8 — Full mock-interview day + gap drain

Theme: simulate the real thing twice, fix what breaks, then stop adding material.

Block 1 (60 min) — Mock 1, full length, recorded

- Ask a friend/colleague (ideally a senior engineer) to play him using DD §13's flow and the question bank in DD §4.15 (give them the Staff/Principal/Fellow tiers). If no one is available, record yourself answering a randomized set: 1 intro, 3 resume drill-downs, 1 whiteboard, 2 on-call, 1 leadership, your questions.
- **Must include these rehearsed answers:** 1. The 90-second intro ordered by his expertise (DD §13, min 0–5). 2. **The observability answer in his vocabulary**: "How would you debug why an agent did something wrong for one customer?" — traceId propagated through retrieval → plan → approval pause (persisted in the checkpoint) → execution → write-back; `tenantId`, `requestingUserId`, `actingIdentity`, `upstreamTrigger` on every span; the query; what it caught. (DD §0.2 Bridge 2, §4.8, §9.) 3. **The IAM answer**: "How does your agent act with the user's permissions?" — ResolveFlow's OAuth chain, the weaknesses you'd fix (GitHub App, encryption, no fallback, no token in checkpoints), and how GitLab's composite identity is the stronger version (DD §0.2 Bridge 1, §7.1). 4. The R8 benchmark question — whichever path you chose, delivered calmly. 5. "Access management isn't SDLC." 6. **At least three infra probes, asked as sudden pivots right after a strong agent answer** (that's how the CrowdStrike round turned): AWS deployment ("which compute, why, who configured it?"), Kubernetes ("what requests/limits did you set and why?"), CI/CD rollback, the "~40%" methodology, Data Cloud freshness. Score each one on whether you **stated your role (OWN / PATTERN / THIN) in the first sentence**.

Block 2 (45 min) — review the recording

- Score each answer: did you give mechanism → example → failure mode → signal? Any adjectives where numbers belonged? Any bluff? Any monologue over 90 seconds without a check-in? Add fixes to `GAPS.md`.

Block 3 (90 min) — drain `GAPS.md`

- Work the list top-down. If an item can't be fixed in 15 minutes, write the honest one-sentence "here's what I know and how I'd find out" answer instead.

Block 4 (60 min) — Mock 2, different emphasis

- Focus on design + assumption removal: run DD §8.1 or §8.6, and have the interviewer remove one assumption every 10 minutes ("no human approval," "100× tenants," "the issue author is hostile," "the queue delivers twice"). Practice re-deriving calmly.

Block 5 (45 min) — final reads

- Re-read `pdf/medium_post.md` once more, fast (second pass). It's your voice; it should feel like yours on Friday.
- Re-read DD §0.3 (audit) and confirm every ⚠️ row is resolved or has a rehearsed answer.

Hard stop by early evening. No new material after this.

End-of-day self-check:

1. Give the 90-second intro cold.
2. Give the observability answer and the IAM answer cold, each under 2 minutes.
3. Pick any number on your resume at random — method, baseline, interval, attribution.
4. Pick three questions to ask him (DD §12).

Fri Oct 9 — Interview day: warm-up only, no new material

Morning (60–90 min max):

- Read DD §0.1–0.2 (who he is, your three bridges).
- Read DD §4.14 (the 24 lessons) and DD §4.16 (litmus test) aloud once.
- Skim `NUMBERS.md` — every number with its method.
- Read your filled-in DD §14.1 (infra ratings + boundary sentences) and DD §14.6's "if asked X, say Y" table once.
- Glance at DD §1.3's pipeline diagram and DD §1.10's loop diagram; draw each once on paper.
- Say the 90-second intro once, out loud.

One hour before:

- Open (in tabs, for reference only if asked to share): your ResolveFlow repo on GitHub (key files: `graph/build.py`, `graph/nodes/await_approval.py`, `graph/nodes/execute.py`, `graph/nodes/independent_review.py`, `app/main.py`, `eval/harness.py`), a blank whiteboard tool, a blank code editor.
- Water, quiet room, camera and audio check.
- Don't read anything new. Don't re-open the question bank.

Reminders for the room:

- Mechanism → example (code/query) → failure mode → signal.
- Infra question → **role first (OWN / PATTERN / THIN)**, then one level deeper on what you owned.
- 30 seconds, then stop and let him choose the drill.
- When you don't know: name the unknown, state the assumption, say how you'd find out.
- Run the litmus test (DD §4.16) out loud when defending a design.
- Ask the calibration question early if invited: "What does your team own today, and where's the biggest gap between the Duo Agent Platform vision and what's shipped?"

Appendix — reading list by topic (with where each fits)

Topic	Source	Day
Your own architecture synthesis	`pdf/medium_post.md`	Oct 2, Oct 8
Vendor-neutral agent security/architecture	`study_guide_enterprise_system_v2.md`	Oct 6
Salesforce internals	`ANIRUDDHA_SINGH_DEEP_DIVE.md` §1, §2.5–2.8, §4, §5	Oct 3, Oct 7
ResolveFlow design notes	`../docs/CHECKPOINTING.md`, `../docs/CAP_THEOREM_AND_AGENTIC_SYSTEMS.md`, `../CHANGELOG.md`	Oct 3
Duo Agent Platform	docs.gitlab.com/user/duo_agent_platform/ (+ /security/, /composite_identity/, /security_threats/)	Oct 5
Duo Workflow runtime	runbooks.gitlab.com/duo-workflow-svc/; handbook duo_workflow design doc + ADRs	Oct 5
GitLab architecture	docs.gitlab.com/development/architecture/	Oct 4
Sidekiq at GitLab	docs.gitlab.com/development/sidekiq/; github.com/sidekiq/sidekiq/wiki/Best-Practices	Oct 4
Ruby basics	ruby-lang.org quickstart; "To Ruby From Python"	Oct 4
Rails basics	guides.rubyonrails.org: Getting Started, Active Record Basics, Query Interface (locking)	Oct 4
GitLab source	gitlab.com/gitlab-org/gitlab (`app/services`, `app/policies`, `app/workers`, `lib/api`, search `duo_workflows`)	Oct 4
Merge trains, Runner, Gitaly	docs.gitlab.com/ci/pipelines/merge_trains/; docs.gitlab.com/runner/; docs.gitlab.com/administration/gitaly/	Oct 5
GitLab values & Fellow role	handbook.gitlab.com/handbook/values/; handbook.gitlab.com/job-description-library/engineering/fellow/	Oct 5
LangGraph	langchain-ai.github.io/langgraph (persistence, interrupts, durable execution, Send/Command)	Oct 6
Eval methodology	swebench.com; DD §4.7	Oct 6
Agent observability	opentelemetry.io/docs/specs/semconv/gen-ai/; his post "Observability in a distributed world" (New Relic blog)	Oct 4, Oct 6
Prompt-injection framing	simonwillison.net/2025/Jun/16/the-lethal-trifecta/	Oct 6
MCP	modelcontextprotocol.io/specification (authorization)	Oct 6
Infra/deploy/observability depth	DD §14; ANIRUDDHA §1.4a, §3; `OFFICE_CLAUDE_QUESTIONS.md`	Oct 2–9 (track)
Kubernetes primitives	kubernetes.io/docs/concepts/ (Workloads → Deployments; Configuration → resource management; HPA)	Oct 6
AWS compute choices	docs.aws.amazon.com: Lambda quotas, API Gateway quotas, ECS developer guide	Oct 5
Distributed locking	martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html	Oct 7

If a URL has moved: search the page title on the same site. Don't spend more than five minutes hunting any one link — the DD sections already contain what you need from each.
